



Editorial

Tasks, test data and solutions for CEOI 2022 were prepared by: Dominik Fistrić, Josip Klepec, Krešimir Nežmah, Ivan Paljak and Paula Vidas. Implementation examples are given in attached source code files.

Task Abracadabra

Prepared by: Krešimir Nežmah and Dominik Fistrić

Preliminary observations

By looking at the numbers written on the face of each card from bottom to top, we can represent the state of the deck at any time by a permutation of the numbers from 1 to n . Denote by `riffle`(L, R) the function which takes as input two arrays of integers L and R , representing the cards in each hand, and returns a new array representing the result of Tin's riffle shuffle on L and R . This procedure can easily be implemented in $O(|L| + |R|)$ by adding the cards one by one from L and R to the result. Therefore, performing one riffle shuffle on a permutation representing the deck can be done in $O(n)$.

After writing out a few examples, one can notice that it seems like the process of shuffling the deck stabilizes at some point. That is, after some number of shuffles we reach a point where shuffling the deck no longer changes it. As we will later see, the number of shuffles needed to reach this point is bounded by n . Therefore, the answer for a query won't change if we set $t = \min(t, n)$. The intended solution for the first subtask is to repeatedly shuffle the deck until we reach this point and store the state of the deck at each point in time. After that, we can answer each query in $O(1)$ by looking up the answer. The total time complexity of such an approach is $O(n^2 + q)$.

Block structure

Let's find a better way to describe how the function `riffle`(L, R) computes the result. We will divide L and R into blocks in the following way: the first block starts at the first element of the array and ends right before the smallest element that is larger than the first one. The second block then starts at this element and ends right before the smallest element larger than it. This process continues until we reach the end of the array. Notice that the maximum value in each block is precisely the front element of the block, and the sequence of maximums of the blocks forms an increasing sequence.

The key observation is the following: instead of computing the result one element at a time, we can compute it block by block, and the final result consists of these blocks placed next to each other, ordered by the front element of the block. Indeed, if the front element of some block is larger than the front element of some other block, then it is also larger than the rest of the elements in that block. Also, the blocks will be sorted in the end because they are sorted initially and at each point the block with the smaller front element is appended to the result. Note that the blocks will never merge together to form a larger block, i.e. if we split the resulting array into blocks, we obtain precisely a permutation of the starting blocks.

Now let's look at a single shuffle operation with this view in mind. Divide the initial permutation into blocks in the same way as described above. After splitting the permutation in half, some of the blocks will be in the left half, some of them will be in the right half, and there will be at most one block which is partially contained in both. If such a block exists, we'll call it the *middle block*, and we'll talk about its left and right parts. Let L and R be the arrays containing the elements of the left and right halves, respectively. If the middle block does not exist, the blocks of L are precisely the blocks of the permutation which are in the left half, and the blocks of R are precisely the blocks from the right half. All blocks from L have a smaller front element than all blocks from R , so after the shuffle the permutation will not change. On the other hand, if the middle block exists, L will have one more additional block, namely, the left part

of the middle block. R might have multiple additional blocks, since the right part of the middle block need not be a block itself. Still, all elements from the right part of the middle block are smaller than the front element of the next block, so the additional blocks in R will form a subdivision into multiple smaller blocks of the right part of the middle block. Once we perform `riffle(L, R)`, the blocks of L and R will become sorted by their front elements. This won't affect the blocks that were fully contained in the right half to begin with. It will, however, affect the blocks which make up the right part of the middle block, because all of their front elements are smaller than the front element of the left part of the middle block. Consequently, these blocks will be moved over somewhere to the left of that left part.

To summarize, we have the following:

- The permutation will stay the same after a shuffle if and only if the middle block does not exist.
- The positions of the blocks which are to the right of the middle block will not change.
- The left part of the middle block is its own block, the right part might have to be split into multiple smaller blocks.
- These smaller blocks will move to the left of the left part of the middle block.

In particular, the middle block splits into at least two smaller blocks, so the total number of blocks increases by at least one after each shuffle. Initially, there is at least one block, and in the end there are at most n blocks, so the total number of shuffles is bounded by $n - 1$.

Challenge for the reader: find a case which achieves the maximum number of shuffles until stabilizing.

Implementation

It is possible to efficiently implement the procedure described above. We store each block using a triplet of integers (v, l, r) , where v represents the value at the front of the block, while l and r are the indices of the ends of the block, from the point of view of the initial permutation. We keep all of the blocks in an `std::set`, to ensure they are sorted according to v at all times. We also keep track of the total length of all the blocks that are currently in the set. Note that if at any time there is a block which is completely contained in the right half, we can remove it from the back of the set, since this block will never again change its position.

When performing a shuffle we do the following:

- While there is a block completely in the right half, remove it.
- If the total length of the blocks in the set is $\frac{n}{2}$, there is no middle block and we can stop.
- Otherwise, the back of the set now contains the middle block. Remove it from the set, split it into smaller blocks and insert them back in the set.

What is left is to figure out how to obtain the initial set of blocks and how to efficiently split the middle block into smaller blocks. For this we precompute for each position i in the initial permutation the position `nxt[i]` representing the smallest index whose corresponding value is larger than the value at position i , or $n + 1$ if there is no such position. This can be done in a standard way in $O(n)$ using a stack. The sequence $i, \text{nxt}[i], \text{nxt}[\text{nxt}[i]], \dots$ determines the starting indices of the blocks starting from i . Using this we can decompose the right part of the middle block in $O(\text{number of new blocks})$.

For the second subtask, all the queries have the same t value, so it is sufficient to run this process until time t , at which point we can iterate over all the blocks in order and obtain the whole array. It is easy to show that the total number of blocks that were in the set at one point or another is at most $2n$, so the total time complexity of obtaining what the array looks like after t shuffles is $O(n \log n)$. After this we can answer all the queries in $O(1)$.



Supporting queries

We can input all the queries, sort them by their t value, and answer them offline. To do this we need to maintain a data structure which keeps track of the currently active blocks, and is able to determine for an arbitrary index i in which block is it currently contained in. This can be done directly with a balanced binary search tree like splay or treap, but we'll describe an easier way involving only a segment tree or a fenwick tree.

First we run the process described above from the second subtask. That is, using a set we repeatedly shuffle the permutation until we obtain the final ordering. We create a list of all the blocks that were contained in the set at some point or another. We order this list according to the front value of each block. Then we create a range-sum, point update segment tree on top of this array of blocks. Each node of the segment tree will store the total length of all active blocks in its range.

We then run this process a second time, starting from the beginning again, but this time we additionally keep track of the lengths of the blocks using the segment tree. Initially, each node of the segment tree stores the value zero, because there are no active blocks. Every time a new block appears in the set, or is deleted from the set, we make it active/inactive in the segment tree as well. That is, we update the point at the corresponding index by adding the length of that block to that position.

To answer a query we have to be able to do the following: for a given value i , determine the index of the first active block such that the prefix sum of the lengths of blocks up to that point is at least i . This is a standard problem which can be solved either in $O(\log^2 n)$ per query using a binary search along with using the segment tree for querying the prefix sums. A better way to do it is to start from the root of the segment tree and directly walk down to the desired index in $O(\log n)$. This solves subtasks 3 and/or 4, depending on the efficiency of the implementation. The total time complexity is $O((n + q) \log n)$.

Task Homework

Prepared by: Krešimir Nežmah and Dominik Fistrić

We can represent a valid expression by a binary tree with n leaves and $n - 1$ inner nodes. The leaves correspond to question marks, and each inner node corresponds to one of the functions **max** or **min**. The problem can now be rephrased as writing a permutation of numbers from 1 to n in the leaves, and propagating these values to the root. The solution for every subtask involves parsing the string from the input and converting it to such a binary tree.

The first subtask can be solved in $O(n! \cdot n)$ by trying out all the different permutations of $\{1, 2, \dots, n\}$, substituting it in the expression and evaluating it.

The second subtask can be solved with a bitmask dp. The state is `dp[node][mask]`, which represents the set of all possible values obtainable in this node if the allowed values are from the mask. For the transition, we try to partition the mask into two submasks (one for each child), in all possible ways. The time complexity is $O(3^n \cdot n^2)$, but in practice it is much faster because all states for which the number of ones in the mask don't match up with the number of nodes in the subtree can be discarded.

There is also a randomized solution which solves the second subtask. We make a guess that the set of obtainable numbers forms an interval (this guess will later turn out to be true). Then we try to evaluate as many different permutations as possible, and store the smallest and largest number we've come across. We declare our final output to be the length of that interval. Challenge for the reader: prove that the probability of success of such an approach is sufficiently high.

The third subtask is solved by looking at the longest sequence of nodes, starting from the root, which are of the same type (all **min** or all **max**). Let's say that there are k of them. The solution is then $n - k$. The proof is left as an exercise to the reader.

Let's look at a node and its subtree. Let S be a set of distinct numbers whose size equals the number of leaves in the subtree. The full solution requires the observation that the set of numbers obtainable in this node, using the numbers from S as the values for the leaves, forms an interval in S . Additionally, we must figure out a way to combine the intervals of the left and right child to get the interval for the node itself. The proof is inductive/recursive and at the same time describes a way to obtain the mentioned intervals, allowing us to solve the problem with a tree dp.

Suppose that a node has a left child with L leaves, and a right child with R leaves. Also, let the interval of possible values for the left child be $[a, b] \subseteq [1, L]$, and $[c, d] \subseteq [1, R]$ for the right child.

If the node is of type **max**, the lower limit for the interval of the node turns out to be $a + c$. Indeed, let's call the smallest $a + c - 1$ numbers from $[1, L + R]$ *small*, and the rest of them *large*. Let's say that the left tree contains x small numbers, and the right subtree contains y small numbers. Since $x + y = a + c - 1$, at least one of $x < a$ and $y < b$ must hold. Without loss of generality assume that $x < a$. Now the a -th smallest number in the left subtree is large, so the maximum of the left and right subtrees will also be large. This shows that the node value is at least $a + c$. This value is also obtainable: put the numbers $[1 + c, L + c]$ in the left subtree, and the rest of them in right subtree. We can make it so that the value of the left subtree turns out to be $a + c$, and the value of the right subtree is c .

If the node is of type **min**, the lower limit will be $\min(a, c)$. This can be shown in a similar way to what is described above. This time, the small numbers will be the ones smaller than $\min(a, c)$, and the rest will be large. The same type of argument shows that the node value is at least $\min(a, c)$. If $a < c$, this can be obtained by putting $[1, L]$ in the left subtree, and $[L + 1, L + R]$ in the right subtree. The case where $a \geq c$ is similar.

We can show a similar thing for the upper limit. If the node is of type **max**, the upper limit is $\max(b + R, d + L)$, and if it is of type **min**, it is $b + d - 1$. To prove that the values in between are obtainable, we just have to slightly alter the way of distributing the numbers in the left and right subtrees, similar to the constructions for achieving the bound. The total time complexity is $O(n)$.



Task Prize

Prepared by: Ivan Paljak and Josip Klepec

Setup

Summarization of the task is somehow choosing the K node labels and then $K - 1$ queries such that they uniquely determine a tree.

Algorithm 1 Choose subset of K node labels

```

 $N \leftarrow$  size of trees  $T_1$  and  $T_2$ 
 $K \leftarrow$  size of subset of node labels to choose
 $P_1 \leftarrow$  preorder of  $T_1$ 
 $S \leftarrow$  first  $K$  node labels with respect to preorder  $P_1$  in tree  $T_1$ 

```

The subset S of node labels forms a connected subgraph in tree T_1 . In tree T_2 it is just some arbitrary subset of nodes.

Algorithm 2 Asking the queries

```

 $P_2 \leftarrow$  preorder of  $T_2$ 
 $s \leftarrow$  array of length  $K$  which values are the values from  $S$  sorted with respect to  $P_2$ 
for  $i = 1, 2, \dots, K - 1$  do
    Ask query for node labels  $s_i$  and  $s_{i+1}$ 
end for

```

Each query may also give some *lowest common ancestor* which is not in S .

We define S_1 and S_2 as subset of nodes in respective trees as union of S and *lowest common ancestors* that appear in queries.

Lemma 0.1. S_i can be formed into a tree structure. Furthermore, S_1 forms a connected subgraph in tree T_1 but it isn't relevant for the rest of the algorithm.

We will describe the algorithm for making such small tree. It can be applied to both T_1 and T_2 .

Algorithm 3 Forming trees

```

 $V \leftarrow S_i$  sorted with respect to preorder of respective tree.
 $nodes \leftarrow$  empty stack
for each  $v \in V$  do
    while stack is not empty AND  $lca(v, stack.top()) \neq stack.top()$  do
        pop from stack
    end while
    if stack is not empty then
        make  $v$  child of the node on top of the stack
    end if
    push  $v$  to top of the stack
end for

```

Now for this trees we want to reconstruct weights of edges. For each tree we have $2 \times (K - 1)$ paths we know the length of. Some may be redundant.

Lemma 0.2. Those paths are such that they uniquely determine the tree.



The proof is not too hard so we won't go into too much detail. Firstly, imagine we construct a graph as described in section for Full Points (see section couple lines below)..

Now, all we need to prove is that that graph is connected. There is no need to prove that the given set of queries don't give contradiction because we can rely on the authentication of the interactor.

We can prove the connectivity by analyzing how we constructed the queries. The connectivity becomes very obvious very quickly after we realize that we first sorted the node labels and then asked of adjacent ones. Every pair of adjacent node labels being connected means the whole set is.

Subtask 3

Even though it is not necessary for full points we can use Gaussian elimination to solve this subtask. Every query forms some equation (with variables being edge weights in respective trees). Now we get two linear systems, one for each tree and we just solve each independently.

https://cp-algorithms.com/linear_algebra/linear-system-gauss.html

Subtask 4

It is not hard to modify algorithm for constructing the small tree to reconstruct the weights as well if we our queries are also sorted by the preorder. But it only holds for the second tree. To achieve this for both trees we need to find a set of node labels such that they can be ordered in a way that they form a monotonic sequence with respect to the preorder in the first tree, but also in the second tree.

One could always choose such subset because $K^2 \leq N$ holds.

If we order node labels with respect to preorder in tree T_1 and look at the sequence of their preordering positions in the tree T_2 . We can find a monotonic subsequence of length K .

Theorem 0.3 (Erdos-Szekeres). *For given positive natural numbers s, r , any sequence of distinct real numbers whose length is at least $N = s r + 1$ contains a monotonically increasing subsequence of length $s + 1$ or a monotonically decreasing subsequence of length $r + 1$ (or both).*

Full Points

Method 1

For a rooted tree we denote $d(x)$ as length of the path that starts at the root of the tree and ends at x .

Then our queries give equations of the form

$$d(x_i) - d(y_i) = w_i$$

Now, we construct a weighted directed graph by adding an edge from y_i to x_i with weight w_i and an edge a reverse edge with negative weight $-w_i$.

Now to find $d(x)$ we just find the length of the path from root to x in our new graph using *dfs*. Such path is guaranteed to exist because of the structure of queries.

When we reconstructed $d(x)$ for every x it is easy to output the answers.

This gives us an algorithm with complexity $\mathcal{O}(K + N \log N)$



Method 2

Alternatively, one could solve the system of equations by using Gaussian elimination faster by observing that each equations forms a path in a tree. Further more a path that from some node to the root of the tree (meaning *lca* of endpoints of the path is always on of them).

Algorithm 4 Algorithm

```
while Tree has more than one vertex do
     $v \leftarrow$  arbitrary leaf that is not the root.
     $eq \leftarrow$  path from that leaf (equation) that is the shortest.
    subtract equation  $eq$  from every other equation that also has leaf  $v$  as endpoint.
    after subtraction equations still form a path, just a different one.
end while
After we have finished the elimination we backtrack to get the desired weights
```

At first this gives us complexity $\mathcal{O}(K^2 + N \log N)$, but it can be sped up to $\mathcal{O}(K \log^2 K + N \log N)$ by keeping the set of equations that begin at every nodes and merging those sets when subtraction equations. If we merge the two sets such that we move everything from smaller set to larger we get the desired complexity. It is convinient to keep equations in stl Set structure to get the minimum path.

Editorial

Tasks, test data and solutions for CEOI 2022 were prepared by: Dominik Fistrić, Josip Klepec, Krešimir Nežmah, Ivan Paljak and Paula Vidas. Implementation examples are given in attached source code files.

Task Drawing

Prepared by: Ante Đerek, Luka Kalinovičić, Paula Vidas, and Dominik Fistrić

Fun fact: This task was originally prepared for CEOI 2013 (also in Croatia). However, it was deemed too difficult, especially because the contestants didn't have full feedback back then, and thus it didn't appear on the contest. Task idea was proposed by Ante Đerek and solution was found by Luka Kalinovičić. To honor them, this year's statement featured Ante and Luka as main characters.

Note that in any tree with maximum degree at most three, we can always find a node with degree at most two. If we root the tree in such a node, every node will have at most two children (i.e. it's a binary tree).

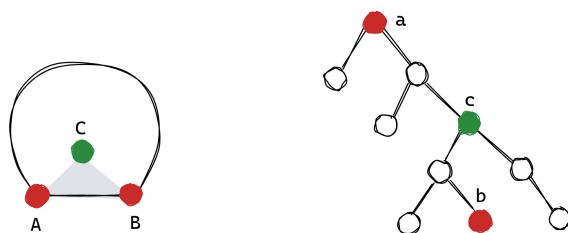
Let's start with the first subtask. Let $P_1, \dots, P_N$ be the given points, such that they form vertices of a convex polygon in that order. Place the root of the binary tree in any of the points, say P_1 . Let k be the size of the subtree of one of its children. We can place this child in node P_2 and decide to place its whole subtree in points $P_2, \dots, P_{k+1}$ in some way. If the node has two children, we place the second child in node P_{k+2} and its subtree in $P_{k+2}, \dots, P_N$. We then solve the subproblems recursively. A careful implementation of this strategy has complexity $O(N \log N)$.

We will now describe a solution for subtasks 2 and 3. Similarly to subtask 1, place the root in some point A on the *convex hull* of the given point set. Sort all other points by the polar angle around point A . In other words, we will sort them such that if point B is before point C , then points A, B, C are ordered counter-clockwise. Again, if k is the subtree size of a child, place this child's subtree in the first k points, choosing the first of the points as the one where the child goes. If there are two children, the other child's subtree is placed in the remaining points, and the child is placed in the first of them.

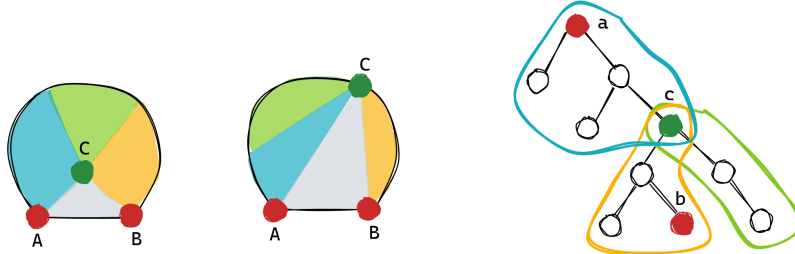
Time complexity of this approach is $O(N^2 \log N)$ if we sort the points each time, which is enough for the second subtask. It can be speed up to $O(N^2)$ if we use a some [selection algorithm](#) with expected complexity $O(N)$ (e.g. `std::nth_element` in C++) to find the k -th smallest element. This is fast enough to pass the third subtask.

Finally, we describe the full solution. Notice that for now we used a procedure that takes a set of points with one distinguished point on the hull, and a tree with one distinguished node, and draws the tree on those points, such that the distinguished node is placed in the distinguished point. It turns out that it's possible to design a procedure that does the same but given two pairs of distinguished points and nodes, such that the points lie on the hull and are adjacent.

The high level idea is the following: If we're given one point-node pair (A, a) , such that A is on the hull and the tree is rooted in a , then we choose some leaf b and one of the adjacent points on hull as B , and (B, b) is our second pair, so we're in the two pair case. Assume now we're given two point-node pairs (A, a) and (B, b) , such that a is the root of the tree, while b is a leaf, and A and B are two adjacent points on the hull. Take c to be some node on the path between a and b , and take C to be a point such that the gray triangle ABC doesn't contain any other points.



Then, we can construct three subproblems, blue, green, and yellow, shown in the figure below. The blue, green, and yellow regions are chosen such that the number of points in each region matches the size of the respective tree, and the points in a region form a consecutive subsequence if we sort all points by polar angle around C . We distinguish two cases, when C is not on the hull and when it is.



It can be proven that there always exists a “good” point C , i.e. a point such that the gray region has no other points inside, AC is a line segment on the hull of the blue region, C is on the hull of the green region, and BC is a line segment on the hull of the yellow region. This is left as an exercise for the reader.

The three subproblems can be solved recursively. In the blue problem we’re given point-node pair (A, a) as root and (C, c) as leaf, in the yellow problem we’re given point-node pair (C, c) as root and (B, b) as leaf, and in the green problem we’re given a point-node pair (C, c) as root.

Now it’s time to analyse the time complexity.

When choosing nodes b or c , it’s not optimal to simply take any node. When we’re given root a and need to choose a leaf b , we’ll do it this way: start from the root, and at each step move down to the “heavy” child, i.e. the one with the larger subtree size, until a leaf is reached. When we’re given root a and leaf b , for node c we’ll take the midpoint of the path between a and b .

Consider a two point-node pair subproblem with N nodes. Let k denote the length of the path from a to b , and let’s look at the subtrees attached to this path that belong to the “light” children. Denote the sizes of these subtrees as $N_1, N_2, \dots, N_k$. Notice that $N_1 + N_2 + \dots + N_k \leq N$, and $N_i \leq \frac{N}{2}$ for all $i = 1, 2, \dots, k$, because they are the light children. Note that each subproblem can be solved in time that is linear in the size of the subproblem (not counting the recursive calls). Therefore, after $\log N$ steps of the process described above, we’ll reach all the subproblems of size N_i , and will have spent $O(N \log N)$ time up that point. Consequently, if $T(N)$ denotes the time required to solve a subproblem of size N , we obtain

$$T(N) = \sum_{i=1}^k T(N_i) + O(N \log N).$$

This recurrence relation works out to be $T(N) = O(N \log^2 N)$.

If in each subproblem we sort the points by angle, a factor of $\log N$ is added to the complexity, and that solution is fast enough for subtask 4. However, sorting is not necessary if we use `std::nth_element`, and that solution should achieve full score for this task.

Task Measures

Prepared by: Ivan Paljak, Paula Vidas, and Dominik Fistrić

Consider the situation when there are N people standing at coordinates $a_1, \dots, a_N$, and without loss of generality let the coordinates be sorted, i.e. $a_1 \leq \dots \leq a_N$.

It's easy to see that there exists an optimal rearrangement after which the order of the people remains the same as the initial order. Otherwise, if there is a moment when some two people swap places in the order, the total time won't increase if they instead don't change their relative order at that moment (but instead move where the other person would have moved).

Let $t_{i,j} = \frac{1}{2}((j-i) \cdot D - (a_j - a_i))$, for any $i \leq j$, and let $t = \max_{i \leq j} t_{i,j}$. We claim that t_{opt} is equal to t .

Clearly, t is a lower bound. After t' seconds, the distance between people i and j can be at most their initial distance $a_j - a_i$ plus $2t'$. Thus $a_j - a_i + 2t_{\text{opt}} \geq (j-i) \cdot D$, which implies $t_{\text{opt}} \geq \frac{1}{2}((j-i) \cdot D - (a_j - a_i)) = t_{i,j}$.

Now we prove t is an upper bound. Consider the following greedy strategy. Let $b_1, \dots, b_N$ be the final coordinates, calculated as: $b_1 = a_1 - t$, and $b_i = \max(a_i - t, b_{i-1} + D)$ for $i > 1$. First, we prove that nobody moved more than t . For some person j , let $i \leq j$ be the maximal index such that $b_i = a_i - t$. Then, we can see that $b_j = b_i + (j-i) \cdot D$, so $b_j - a_j = (j-i) \cdot D + b_i - a_j = (j-i) \cdot D + a_i - t - a_j \leq 0$. On the other hand, $b_j \geq a_j - t$, i.e. $b_j - a_j \geq -t$, so we're done. Second, we prove that after the rearrangement, no two consecutive people are standing closer than D . This follows from $b_i \geq b_{i-1} + D$, and the proof is complete.

Naively calculating all $t_{i,j}$ and taking the maximum one, for each of the M scenarios, is enough to solve the first subtask. Time complexity is $O(M(N+M)^2)$.

To solve the second subtask, we don't need the formula. Instead, we can binary search t_{opt} directly. To check if some t' is feasible, we use the greedy strategy from the proof above, and at the end check if all consecutive distances are at least D . Time complexity is $O(M(N+M) \log T)$, where T is the maximum possible value of t_{opt} given the constraints.

The third subtask can be solved using the formula for t . We can rewrite $t_{i,j} = \frac{1}{2}((a_i - iD) - (a_j - jD))$. Since each new person is added to the end of the line, it's enough to maintain the current maximum value of $a_i - iD$. Then, when j -th person is added, it's easy to find the maximal $t_{i,j}$ over all i . Time complexity is $O(N+M)$.

In order to solve the fourth subtask, we need to be able to insert a person at an arbitrary place in the line. Consider the value $a_i - iD$. When a person is inserted somewhere in the middle of the line, this value remains the same for everyone on the left side, and decreases by D for everyone on the right side of this person. Furthermore, values $t_{i,j}$ remain the same between people who are on the same sides relative to the new person, and it increases by D between people on opposite sides. Therefore, to find the new t_{opt} , it's enough to find the maximum of $a_i - iD$ in the left part and the minimum in the right part.

This process can be simulated using a segment tree which supports adding a number to a range and querying for minimum and maximum of a range. This is a classic problem, for more info check this [link](#). Time complexity is $O((N+M) \log(N+M))$.

Task Parking

Prepared by: Ivan Paljak and Krešimir Nežmah

The first subtask can be solved in various ways. Since M is small, it's possible to go over all the cases systematically with pen and paper. Alternatively, we can have a brute force solution which uses BFS to go over the state space.

If at any point there are two vehicles of the same color in the same parking spot, we can ignore this parking spot for the rest of the process since these vehicles are in the correct spot. If two vehicles of the same color are not initially in the same parking spot, then we obviously need to spend at least one move for this color. Additionally, if both of these vehicles are top vehicles, we clearly need at least two moves. As we'll see later, there is one more case where we have to use an additional move (one of the cycle cases). We will present a solution which constructs a sequence of moves that achieves this bound, in case a solution exists.

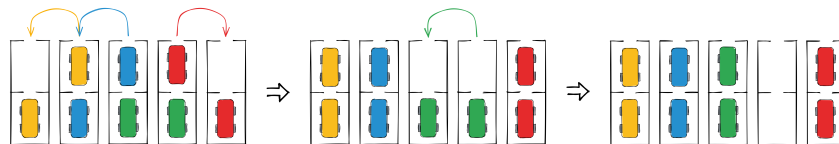
Each parking spot contains two positions (the bottom one and the top one). We'll create a graph with $2M$ nodes representing the parking spot positions. The edges are defined as follows:

- if two vehicles of the same color are in different parking spots, we connect their positions,
- if a parking spot is full, we connect the bottom and the top position.

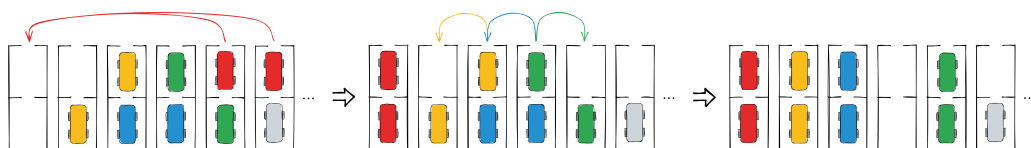
Each connected component of such a graph will be either a cycle, or a chain, where the ends of the chain are two bottom vehicles. If a connected component has two adjacent top nodes, we'll call such a pair a *top pair* for this connected component. Note that a top pair corresponds to two top vehicles of the same color in different parking spaces. We define the term *bottom pair* analogously.

We'll analyze chains and cycles separately. It turns out that each chain can be solved with at most one additional empty parking spot. Similarly, each cycle requires at least one empty parking spot and each cycle can be solved with at most two empty parking spots.

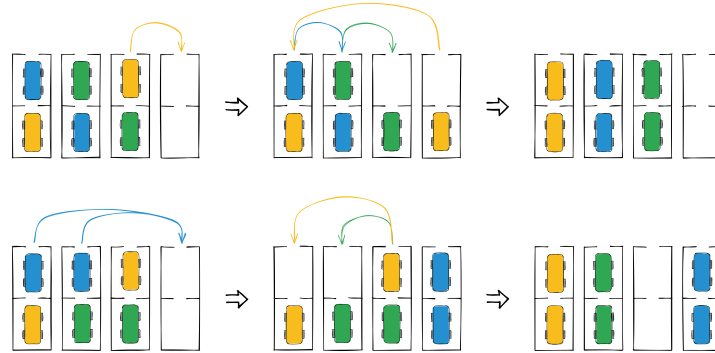
Observation 1: A chain with no top pair can be solved with no additional empty parking spots. Note that every chain must contain at least one bottom pair. In this case there is exactly one. The image below depicts how to solve such a chain.



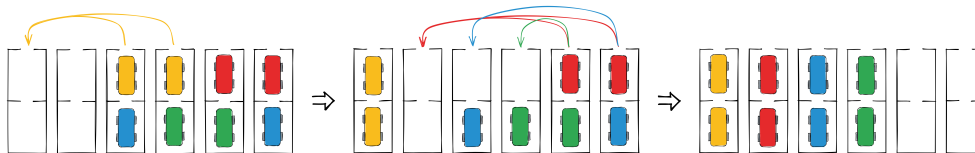
Observation 2: A chain with a top pair requires at least one additional empty parking spot. Furthermore, one parking spot is sufficient. Starting from one end of the chain, we can find the first time a top pair appears and move this top pair to the empty parking spot. We are now left with two chains, where one of them requires no additional parking spots. We first solve this chain, giving us an additional empty parking spot, and then continue solving the other chain. The image below depicts this process.



Observation 3: A cycle with at most one top pair can be solved with only one additional empty parking spot. If there are no top pairs, then the top vehicles of the cycle are a permutation of the bottom ones. We can solve this case by removing any top vehicle to an empty parking spot, reducing the problem to a chain as in observation 1. If the cycle contains exactly one top pair, it contains exactly one bottom pair as well. We move the top pair to an empty parking spot, reducing the problem to a chain as in observation 1.



Observation 4: A cycle with more than one top pair requires at least two empty parking spots. We can take any top pair, move it to an empty parking spot, reducing the problem to a chain as in observation 2.



Combining these observations yields a full solution. Note that we should prioritize solving chains, to solving cycles, because after solving a chain we gain an additional empty parking spot. Similarly, we should prioritize chains and cycles with a smaller number of top pairs.

The subtasks were intended to be solved by using only a subset of these observations, or by alternative approaches. Everything mentioned above can be implemented in $O(N)$ or $O(N \log N)$. The subtasks where $N \leq 1000$ allow for slower solutions with easier implementation.

Subtask 2 can be solved by first moving all top pairs to empty parking spots. There will always be enough space because $2N \leq M$. After that, we are left with only chains and cycles that can be solved as described in observations 1 and 3.

Subtasks 3 and 4 contain no chains. The following greedy approach works for this case:

- if at some point we can pair a vehicle with another one of the same color, we do it,
- otherwise, if there is a top pair, move it to an empty parking spot,
- otherwise, move any vehicle from the top to an empty parking spot.

If at any point there are no parking spots available, there is not solution. Note that this approach does not work for the other subtasks because, we should prioritize moving top pairs from chains.